

INSTITUTO FEDERAL DO RIO GRANDE DO SUL - IFRS
CAMPUS ERECHIM

EMANUEL D. TONIAL
GABRIEL H. SCHAEFFER

Pesquisa sobre ferramentas de Desenvolvimento Mobile

ERECHIM
2026

1. Estratégias de Desenvolvimento

1.1 Desenvolvimento nativo

Na abordagem de desenvolvimento nativo, o aplicativo é construído com tecnologias próprias de cada plataforma, o que permite aproveitar ao máximo os recursos do sistema operacional.

No Android, por exemplo, as tecnologias nativas são Kotlin e Jetpack Compose. O ecossistema Apple é composto por Swift e SwiftUI.

Portanto, o desenvolvimento nativo é o que permite melhor integração com as APIs e recursos do sistema, o que significa que o desenvolvimento pode parecer mais “natural”.

Para usar, os usuários precisam instalar a aplicação no dispositivo escolhido.

1.2 Desenvolvimento híbrido

O desenvolvimento híbrido busca combinar vantagens do desenvolvimento web e do desenvolvimento nativo. Fazendo o desenvolvimento dessa forma, uma única base de código pode ser utilizada para fazer aplicativos para Android e iOS.

Essencialmente, existem dois tipos de desenvolvimento híbrido, o baseado em WebView, onde o aplicativo é uma aplicação web rodando em um contêiner nativo. Como exemplos desta, temos Ionic e Cordova; e o Cross Platform com renderização nativa, onde o código é compartilhado entre as plataformas, mas a interface gráfica é renderizada usando componentes nativos ou um motor gráfico próprio. A título de exemplo dessa categoria, podemos citar o React Native e o Flutter.

Para usar, os usuários precisam instalar a aplicação no dispositivo escolhido.

1.3 Desenvolvimento web

O aplicativo é criado utilizando tecnologias de desenvolvimento web, e é executado no navegador do dispositivo.

As principais tecnologias utilizadas são o HTML, CSS e JavaScript, com ferramentas como React, Vue, Angular, etc.

O usuário não precisa baixar nada; ele acessa o aplicativo no navegador por meio de uma URL.

Uma evolução mais recente dessa estratégia são os *Progressive Web Apps*, que permitem instalar o sistema na tela inicial e usar alguns recursos offline.

2. Ferramentas

2.1 SwiftUI (nativo, iOS)

SwiftUI é o framework declarativo da Apple para criar interfaces em todo o seu ecossistema, como iOS, iPadOS, macOS, watchOS e visionOS. Ele foi lançado em 2019 e hoje é uma das principais escolhas para projetos novos na plataforma Apple. Em vez de montar e atualizar as telas manualmente, como acontecia no UIKit, o desenvolvedor descreve como a interface deve aparecer, e o próprio framework atualiza a tela quando o estado do app muda.

Vantagens: sintaxe simples; pré-visualização em tempo real no Xcode; integração nativa com o ecossistema Apple; desempenho muito bom por ser a própria solução oficial da plataforma.

Desvantagens: funciona apenas em plataformas Apple; alguns recursos mais novos exigem versões recentes do sistema; possui menos bibliotecas de terceiros do que frameworks multiplataforma.

Quem desenvolve: a Apple, que atualiza o framework a cada WWDC.

Quem usa: os aplicativos da própria Apple e a maior parte dos apps novos publicados para iOS, iPadOS e visionOS em 2026.

2.2 Jetpack Compose (nativo, Android)

Jetpack Compose é o toolkit declarativo do Google para criar interfaces Android usando Kotlin, substituindo o antigo sistema de layouts em XML. Nele, funções anotadas com `@Composable` descrevem a interface a partir do estado da aplicação, e o framework cuida da renderização e das atualizações da tela. Em 2026, o compilador K2 do Kotlin 2.2 deixou a inferência de estabilidade mais inteligente, reduzindo recomposições desnecessárias sem exigir anotações manuais. O Kotlin também já é a linguagem padrão em 100% dos novos apps Android do próprio Google, e o Compose é recomendado oficialmente como ferramenta de UI.

Vantagens: muito menos código repetitivo do que o sistema antigo em XML; animações e temas modernos, como Material You, ficam mais fáceis de implementar; funciona bem com Kotlin Multiplatform para compartilhar lógica com iOS.

Desvantagens: quem vem do sistema antigo de Views/XML costuma sentir curva de aprendizado; entender recomposição exige um novo jeito de pensar a interface; fora do Android, seu uso é limitado sem apoio de outras tecnologias.

Quem desenvolve: o Google, como parte do Android Jetpack.

Quem usa: Airbnb, Spotify, Meta e partes do próprio Google Play já usam Compose em produção.

2.3 React Native (híbrido, sem WebView)

React Native é um framework de código aberto criado pela Meta em 2015. Ele permite escrever a lógica e a interface em JavaScript ou TypeScript e transformá-las em componentes nativos reais no iOS e no Android, sem depender de WebView.

Vantagens: usa um único código-base em JavaScript/TypeScript para iOS e Android; possui uma comunidade enorme e muitas bibliotecas; renderiza componentes nativos de verdade, e não uma página dentro do navegador.

Desvantagens: algumas funções muito específicas ainda exigem código nativo em Swift ou Kotlin; aplicativos com gráficos muito pesados continuam mais adequados ao desenvolvimento nativo puro; migrar projetos antigos para a nova arquitetura pode dar bastante trabalho.

Quem desenvolve: a Meta, agora com governança da React Foundation e da Linux Foundation, com apoio de empresas parceiras como a Callstack.

Quem usa: Facebook, Instagram e Messenger, além de Microsoft, Shopify e Discord.

2.4 Ionic + Capacitor (híbrido, com WebView)

Ionic é um framework de interface open source para criar aplicativos multiplataforma usando HTML, CSS e JavaScript ou TypeScript. Ele pode ser combinado com Angular, React, Vue ou Svelte. Desde 2019, o Ionic usa o Capacitor, um runtime nativo criado pelo próprio time como sucessor do Apache Cordova, para empacotar a aplicação web dentro de um WebView em iOS e Android.

Vantagens: reaproveita o conhecimento de equipes que já trabalham com web; permite gerar app para iOS, Android e também PWA a partir da mesma base; reduz custos quando já existe uma aplicação web pronta.

Desvantagens: como a interface roda dentro de uma WebView, aplicativos com uso intenso de GPU, como jogos 3D ou edição de vídeo em tempo real, sofrem mais com essa camada extra; também depende de plugins para vários recursos nativos.

Quem desenvolve: a Ionic, empresa fundada em 2013 e responsável pelo Ionic e pelo Capacitor.

Quem usa: Sworkit, Shipt, Untappd, MarketWatch e Nationwide são exemplos conhecidos de apps feitos com Ionic.

2.5 Vue (web)

Vue é um framework JavaScript criado por Evan You em 2014 para desenvolver interfaces web de forma simples e progressiva. Ele usa HTML, CSS e JavaScript, organizando a aplicação em componentes reutilizáveis. Uma de suas principais características é permitir adoção gradual: ele pode ser usado apenas em uma parte de um site já existente ou em uma aplicação completa.

Em 2026, o Vue continua sendo uma das opções mais conhecidas para projetos web, especialmente quando o time quer simplicidade, organização e uma curva de aprendizado mais leve.

Vantagens: é fácil de aprender; tem documentação muito boa; usa componentes reutilizáveis; pode ser adotado aos poucos em projetos já existentes.

Desvantagens: tem ecossistema menor que o React; possui menos vagas e menos bibliotecas do mercado; em times muito grandes, alguns preferem frameworks com mais convenções já definidas.

Quem desenvolve: o desenvolvedor Evan You e outros colaboradores e patrocinadores do projeto.

Quem usa: GitLab, Wikimedia Foundation, NASA, Adobe, Alibaba e várias aplicações web modernas usam Vue em produção.

2.6 Flutter (híbrido, sem WebView)

Flutter é o kit de ferramentas de UI open source do Google, lançado em 2015 e estabilizado como versão 1.0 em 2018. Ele usa a linguagem Dart e desenha toda a interface com seu próprio motor gráfico, sem depender de WebView nem dos componentes nativos do sistema. Isso permite controlar a aparência da aplicação com muita precisão a partir de uma única base de código para Android, iOS, web e desktop.

Vantagens: desempenho próximo do nativo; hot reload acelera o desenvolvimento e os testes visuais; uma única base pode atender mobile, web e desktop.

Desvantagens: os aplicativos costumam ficar um pouco maiores do que equivalentes nativos, por causa do motor de renderização embutido; novas APIs de plataforma podem demorar um pouco para chegar; aprender Dart é uma barreira extra para quem vem de outras linguagens.

Quem desenvolve: o Google.

Quem usa: BMW, Google, Toyota e Alibaba, entre outras empresas.

2.7 .NET MAUI (híbrido, sem WebView)

.NET MAUI (Multi-platform App UI) é o framework multiplataforma da Microsoft e sucessor do Xamarin.Forms, cujo suporte oficial já foi encerrado. Ele permite escrever a lógica em C# e a interface em XAML, usando um único projeto para Android, iOS, macOS e Windows (ZCO,

2026). Sua arquitetura usa “handlers”, que fazem o mapeamento de cada controle para o elemento nativo real de cada plataforma, substituindo o sistema antigo de “renderers” do Xamarin, que era mais pesado e mais preso a detalhes de cada sistema.

Vantagens: um único projeto para várias plataformas; Hot Reload funciona para XAML, C#, recursos e assets; integração muito boa com o ecossistema .NET, inclusive com Blazor em alguns cenários.

Desvantagens: depende fortemente do ecossistema .NET; para quem não conhece C#, a curva de aprendizado pode ser maior; em alguns casos, o desempenho ainda fica atrás do nativo puro ou do Flutter.

Quem desenvolve: Microsoft.

Quem usa: principalmente empresas que vieram do Xamarin.Forms e equipes corporativas que já trabalham com .NET no backend.

2.8 Apache Cordova (híbrido, com WebView)

O Cordova empacota uma aplicação HTML, CSS e JavaScript dentro de uma WebView nativa e oferece plugins para acessar câmera, GPS e notificações. Ele foi a base histórica do Ionic até 2019, quando o time do Ionic passou a adotar o Capacitor como runtime recomendado.

Vantagens: é simples para quem já sabe HTML, CSS e JavaScript; possui bastante documentação e muitos plugins acumulados ao longo dos anos; ainda serve para apps simples que só precisam empacotar conteúdo web.

Desvantagens: perdeu espaço para Capacitor, React Native e Flutter; o projeto depende de poucos voluntários; a adaptação a mudanças recentes da Apple e do Google pode demorar mais.

Quem desenvolve: a Apache Software Foundation, como projeto comunitário.

Quem usa: aplicações corporativas antigas e projetos simples que ainda não migraram para o Capacitor.

2.9 Kotlin Multiplatform + Compose Multiplatform (nativo, com código compartilhado)

Kotlin Multiplatform (KMP) é uma tecnologia open source da JetBrains que permite compartilhar código, principalmente a lógica de negócio, entre Android, iOS, desktop, web e backend, sem perder os benefícios do desenvolvimento nativo em cada plataforma.

Quando combinado com o Compose Multiplatform, também da JetBrains e baseado no Jetpack Compose do Google, é possível compartilhar até a interface entre Android e iOS, compilando tudo para componentes nativos reais via Kotlin/Native. No KotlinConf 2026, JetBrains e Google mostraram o Compose Multiplatform rodando em produção em empresas como a Sony.

Vantagens: permite reaproveitar a lógica de negócio e, em alguns casos, a própria interface; mantém a compilação nativa em cada plataforma; é uma boa solução para projetos que já existem e querem compartilhar parte do código.

Desvantagens: a interoperabilidade com Swift ainda está evoluindo; o suporte estável completo em iOS ainda está amadurecendo; o alvo web do Compose Multiplatform ainda está em beta; equipes focadas em Swift e iOS precisam aprender Kotlin.

Quem desenvolve: a JetBrains, com apoio do Google na parte Android.

Quem usa: Netflix, McDonald's, Cash App, Duolingo, Forbes, Philips, Sony e H&M, entre outras.

2.10 WebAssembly aplicado a apps web e PWAs (web)

WebAssembly (Wasm) é um padrão aberto que define um formato binário portátil e uma máquina virtual baseada em pilha, permitindo executar código escrito em linguagens como Rust, C, C++ ou Go com desempenho próximo ao nativo dentro do navegador. Diferente de outras tecnologias desta seção, o Wasm não substitui HTML, CSS e JavaScript. Ele complementa a aplicação web, inclusive em PWAs, ajudando em tarefas pesadas como edição de imagem e vídeo, renderização 3D e inferência de modelos de IA no próprio dispositivo. Em muitos casos, JavaScript sozinho pode ser bem mais lento nessas tarefas.

Vantagens: aproxima aplicações web do desempenho de apps nativos em tarefas pesadas; roda em sandbox, o que melhora a segurança; é portátil entre navegadores, dispositivos e até ambientes de edge e serverless, via WASI.

Desvantagens: não compensa para aplicações simples, como sistemas CRUD, em que o gargalo costuma ser rede ou banco de dados; adiciona complexidade ao processo de build; as linguagens mais maduras para Wasm, como Rust, C e C++, exigem conhecimento de programação de sistemas.

Quem desenvolve/mantém: o padrão é definido pelo W3C WebAssembly Community Group e implementado nos motores dos principais navegadores.

Quem usa: Figma, Unity WebGL, Adobe Premiere Rush e Google Earth são exemplos de aplicações que rodam no navegador com ajuda do Wasm.

3. Comparativo e Discussão

Ao comparar as dez ferramentas dentro das três estratégias de desenvolvimento, fica claro que não existe uma opção “melhor” em todos os casos. A escolha depende do tipo de projeto e do que ele precisa. De modo geral, as ferramentas nativas, como SwiftUI e Jetpack Compose, oferecem o melhor desempenho e a melhor integração com o sistema operacional, mas exigem manter uma base de código para cada plataforma. As ferramentas híbridas sem WebView, como React Native, Flutter, .NET MAUI e Kotlin Multiplatform com Compose Multiplatform, tentam equilibrar bom desempenho com compartilhamento de código. Já as híbridas com WebView, como Ionic + Capacitor e Apache Cordova, aproveitam mais o conhecimento de desenvolvimento web, mas podem perder desempenho em apps mais pesados. As estratégias web, como PWA e WebAssembly, têm maior alcance e facilitam a distribuição, mas têm limitações no acesso a recursos nativos, principalmente no iOS.

Em termos de desempenho e aparência, existe uma ordem geral. SwiftUI e Jetpack Compose ficam no topo, porque são frameworks nativos. Depois vêm Flutter e React Native, que não dependem de WebView, mas seguem caminhos diferentes para renderizar a interface. Abaixo deles estão Ionic + Capacitor e Apache Cordova, que sofrem mais com a camada web. O PWA fica mais abaixo nessa comparação, e é justamente por isso que o WebAssembly é útil: ele ajuda a executar tarefas mais pesadas no navegador com desempenho mais próximo do nativo.

No custo de desenvolvimento e no alcance de plataformas, a situação muda um pouco. Um único código-base em React Native, Flutter, Ionic, Cordova, .NET MAUI ou PWA pode atender Android, iOS e, em alguns casos, web e desktop. Isso reduz o esforço de manter versões separadas. Já o desenvolvimento nativo tradicional exige mais trabalho, porque normalmente é preciso manter soluções diferentes para Android e iOS. Nesse ponto, o Kotlin Multiplatform ajuda, pois permite reaproveitar a lógica do aplicativo sem abrir mão do binário nativo em cada plataforma. A escolha também costuma depender da equipe: times que já trabalham com JavaScript ou TypeScript tendem a preferir React Native, Ionic ou Cordova; times de C# e .NET costumam escolher .NET MAUI; e

times que já usam Kotlin e querem alcançar iOS sem trocar de linguagem tendem ao Kotlin Multiplatform.

Na parte de distribuição e manutenção, o PWA se destaca por não depender da aprovação das lojas de aplicativos e por evitar taxas dessas plataformas, já que pode ser instalado diretamente pelo navegador. Em troca, ele perde parte do acesso aos recursos nativos e não tem a mesma presença das lojas oficiais. As soluções nativas e híbridas, por outro lado, precisam passar pelas lojas, mas oferecem acesso mais completo ao dispositivo. A maturidade do ecossistema também influencia bastante: o Apache Cordova mostra o risco de apostar em uma tecnologia com comunidade em queda, enquanto Flutter e React Native continuam crescendo e sendo usados em projetos grandes.

Portanto, não existe uma ferramenta ideal para todo projeto. A escolha depende de fatores como desempenho, tamanho da equipe, orçamento, prazo, necessidade de publicação nas lojas e tempo de manutenção esperado. O Quadro 1 resume, para cada uma das dez ferramentas, a estratégia a que pertence e o tipo de projeto em que ela costuma ser mais vantajosa.

REFERÊNCIAS

AMAZON WEB SERVICES (AWS). What's the difference between web apps, native apps, and hybrid apps? Seattle: Amazon Web Services, [s.d.]. Disponível em: <https://aws.amazon.com/pt/compare/the-difference-between-web-apps-native-apps-and-hybrid-apps/>. Acesso em: 5 ago. 2026.

APPLE. WWDC26 SwiftUI guide. Apple Developer, jun. 2026. Disponível em: <https://developer.apple.com/wwdc26/guides/swiftui/>. Acesso em: 05 ago. 2026.

ASKANTECH. WebAssembly Performance, Real Use Cases and When to Deploy It in Production (2026). AskAnTech, abr. 2026. Disponível em: <https://www.askantech.com/webassembly-production-2026-performance-use-cases/>. Acesso em: 05 ago. 2026.

ATAKINTERACTIVE. WebAssembly in 2026: The Future of High-Performance Web Applications. Atak Interactive, maio 2026. Disponível em: <https://www.atakinteractive.com/blog/webassembly-2026>. Acesso em: 05 ago. 2026.

DAILYDEV. WebAssembly for Web Developers: Getting Started with Wasm in 2026. daily.dev, maio 2026. Disponível em: <https://daily.dev/blog/webassembly-web-developers-getting-started-wasm/>. Acesso em: 05 ago. 2026.

DEEPClick. Progressive Web Application (PWA): Complete 2026 Guide. DeepClick, jun. 2026. Disponível em: <https://deepclick.com/resources/blog/progressive-web-application-guide/>. Acesso em: 05 ago. 2026.

DIGITALAPPLIED. Progressive Web Apps 2026: Complete Development Guide. Digital Applied, jan. 2026. Disponível em: <https://www.digitalapplied.com/blog/progressive-web-apps-2026-completeddevelopment-guide>. Acesso em: 05 ago. 2026.

DREAMSTECHNOLOGIES. Jetpack Compose vs Traditional Views: Modern UI Development for Android in 2026. Dreams Technologies, jun. 2026. Disponível em: [https://dreamstechnologies.com/jetpack-composevs-traditional-views-modern-ui-development-for-and](https://dreamstechnologies.com/jetpack-composevs-traditional-views-modern-ui-development-for-android-in-2026/) roid-in-2026/. Acesso em: 05 ago. 2026.

EDUONIX. WebAssembly (Wasm) in 2026: Building High-Performance Web Applications Beyond JavaScript. Eduonix Blog, jul. 2026. Disponível em: [https://blog.eduonix.com/2026/07/webassemblywasm-in-2026-building-high-performance-web-applic](https://blog.eduonix.com/2026/07/webassemblywasm-in-2026-building-high-performance-web-applications-beyond-javascript/) ations-beyond-javascript/. Acesso em: 05 ago. 2026.

EMMANUEL, I. What's Actually New in Kotlin Multiplatform and Compose Multiplatform in 2026. Medium, jul. 2026. Disponível em: <https://emmanueldav.medium.com/whats-actually-new-in-kotlin-multiplatformand-compose-multiplat> form-in-2026-0ba10c61ac62. Acesso em: 05 ago. 2026.

FORESIGHTMOBILE. Why Flutter Outperforms React Native and Native Development in 2026. Foresight Mobile, jul. 2026. Disponível em: <https://foresightmobile.com/blog/why-flutter-outperforms-thecompetition>. Acesso em: 05 ago. 2026.

GAREJAKIRIT. The Complete SwiftUI Roadmap for 2026: What Every iOS Developer Must Know. Medium, jun. 2026. Disponível em: [https://medium.com/@garejakirit/the-complete-swiftui-roadmap-for-2026-what-every-ios-developer-](https://medium.com/@garejakirit/the-complete-swiftui-roadmap-for-2026-what-every-ios-developer-must-know-58b2dd9e6145) must-know-58b2dd9e6145. Acesso em: 05 ago. 2026.

GOOGLE. Kotlin for Jetpack Compose. Android Developers, 2026. Disponível em: <https://developer.android.com/develop/ui/compose/kotlin>. Acesso em: 05 ago. 2026.

GROKIPEDIA. Apache Cordova. Grokipedia, mar. 2026. Disponível em: https://grokipedia.com/page/Apache_Cordova. Acesso em: 05 ago. 2026.